Advanced SQL Tasks

A. Creating a Database View

CREATE VIEW EmployeeDetails AS SELECT e.emp_id, e.name AS employee_name, e.salary, d.name AS dept_name, p.project_name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id LEFT JOIN Employee_Project ep ON e.emp_id = ep.emp_id LEFT JOIN Project p ON ep.project_id = p.project_id;

```
Select * from EmployeeDetails;
```

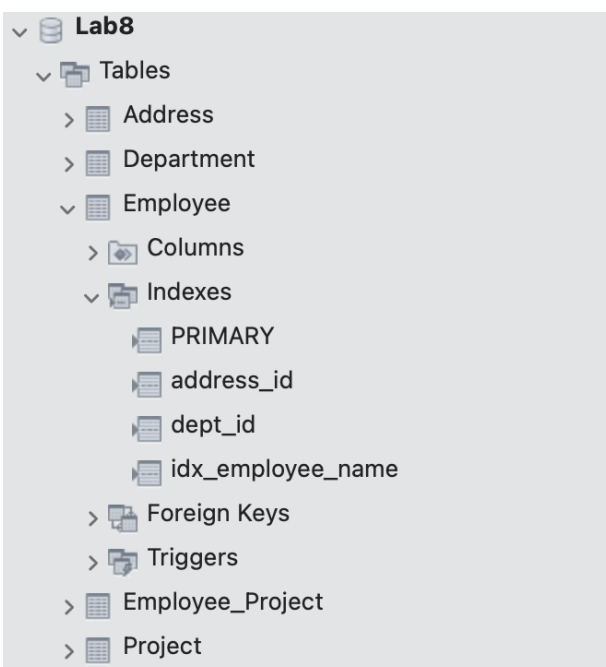
100% 1:2

Result Grid Filter Rows: Search Exp

emp_id	employee_name	salary	dept_name	project_na...
111	Zaineh	100000	Tech	X
111	Zaineh	100000	Tech	Y
112	Yasmeen	160000	Marketing	NULL
113	Mira	140000	Finance	NULL
114	Shimaa	200000	HR	X
114	Shimaa	200000	HR	Z
115	Dean	150000	Tech	X
115	Dean	150000	Tech	Y
115	Dean	150000	Tech	Z

B. Adding an Index

CREATE INDEX idx_employee_name ON Employee(name);



Java Repository Implementation (JDBC)

A. Create Model Class (Employee.java)

```
public class Employee {
    private int empId;
    private String name;
    private double salary;
    private Integer addressId;
    private Integer deptId;

    public Employee() {}

    public Employee(int empId, String name, double salary, Integer addressId,
Integer deptId) {
        this.empId = empId;
        this.name = name;
        this.salary = salary;
        this.addressId = addressId;
        this.deptId = deptId;
    }

    public int getEmpId() {
        return empId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public Integer getAddressId() {
        return addressId;
    }

    public Integer getDeptId() {
        return deptId;
    }
}
```

B. Create Repository Class (EmployeeRepository.java)

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class EmployeeRepository {
    private final String url = "jdbc:mysql://localhost:3306/Lab8";
    private final String user = "root";
    private final String password = "root123456";

    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }

    //Gets all employees.
    public List<Employee> findAll() {
        List<Employee> list = new ArrayList<>();
        String sql = "SELECT * FROM Employee";
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                list.add(new Employee(
                        rs.getInt("emp_id"),
                        rs.getString("name"),
                        rs.getDouble("salary"),
                        rs.getInt("address_id"),
                        rs.getInt("dept_id")
                ));
            }
        } catch (SQLException e) { e.printStackTrace(); }
        return list;
    }

    //Gets an employee
    public Employee findById(int empId) {
        String sql = "SELECT * FROM Employee WHERE emp_id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, empId);
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    return new Employee(rs.getInt("emp_id"),
rs.getString("name"),
                            rs.getDouble("salary"), rs.getInt("address_id"),
                            rs.getInt("dept_id"));
                }
            }
        } catch (SQLException e) { e.printStackTrace(); }
        return null;
    }

    //Inserts a new employee
    public void create(Employee employee) {
        String sql = "INSERT INTO Employee (emp_id, name, salary, address_id,
```

```
dept_id) VALUES (?, ?, ?, ?, ?)";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, employee.getEmpId());
            pstmt.setString(2, employee.getName());
            pstmt.setDouble(3, employee.getSalary());
            pstmt.setInt(4, employee.getAddressId());
            pstmt.setInt(5, employee.getDeptId());
            pstmt.executeUpdate();
            System.out.println("Created employee: " + employee.getName());
        } catch (SQLException e) { e.printStackTrace(); }
    }

    //Updates an employee
    public void update(Employee employee) {
        String sql = "UPDATE Employee SET name = ?, salary = ? WHERE emp_id =
?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, employee.getName());
            pstmt.setDouble(2, employee.getSalary());
            pstmt.setInt(3, employee.getEmpId());
            pstmt.executeUpdate();
            System.out.println("Updated employee ID: " +
employee.getEmpId());
        } catch (SQLException e) { e.printStackTrace(); }
    }

    //Deletes an employee
    public void delete(int empId) {
        String sql = "DELETE FROM Employee WHERE emp_id = ?";
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, empId);
            pstmt.executeUpdate();
            System.out.println("Deleted employee ID: " + empId);
        } catch (SQLException e) { e.printStackTrace(); }
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        EmployeeRepository repo = new EmployeeRepository();

        // 1. Create
        Employee newEmp = new Employee(116, "Phuong Nguyen", 120000, 1, 1);
        repo.create(newEmp);

        // 2. Find All
        System.out.println("All employees: " + repo.findAll().size());

        // 3. Find By Id
        Employee found = repo.findById(116);
        System.out.println("Found: " + (found != null ? found.getName() :
"Not found"));

        // 4. Update
        if (found != null) {
            found.setName("Phuong Updated");
            repo.update(found);
        }

        // 5. Delete
        repo.delete(116);

        // 6. Confirm Deletion
        System.out.println("Final count: " + repo.findAll().size());
    }
}
```